

Introduction of Object Oriented Programming

Object Oriented Programming:

Object oriented programming can be defined as a programming model which is based upon the concept of objects. Objects contain data in the form of attributes and code in the form of methods. In object-oriented programming, computer programs are designed using the concept of objects that interact with real world. Object oriented programming languages are various but the most popular ones are class-based, meaning that objects are instances of classes, which also determine their types.

Languages used in Object Oriented Programming:

Java, C++, C#, Python, PHP, JavaScript, Ruby, Perl, Objective-C, Dart, Swift, Scala.

Difference between Procedural Programming and Object-Oriented Programming:

Procedural Oriented Programming	Object Oriented Programming
In procedural programming, program is divided into small parts called <i>functions</i> .	In object-oriented programming, program is divided into small parts called <i>objects</i> .
Procedural programming follows <i>top down approach</i> .	Object oriented programming follows <i>bottom up approach</i> .
There is no access specifier in procedural programming.	Object oriented programming have access specifiers like private, public, protected etc.
Adding new data and function is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way for hiding data so it is <i>less secure</i> .	Object oriented programming provides data hiding so it is <i>more secure</i> .
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, function is more important than data.	In object-oriented programming, data is more important than function.
Procedural programming is based on <i>unreal world</i> .	Object oriented programming is based on <i>real world</i> .
Examples: C, FORTRAN, Pascal, Basic etc.	Examples: C++, Java, Python, C# etc.

Introduction of Object-Oriented Programming

As the name suggests, Object-Oriented Programming or OOPs refers to languages that use objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

OOPs Concepts:

- Class
- Objects
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

1. Class:

A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type. A class is like a blueprint for an object.

For Example: Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, mileage are their properties.

(a) Class Employee

Characteristics	Methods
Name	computeSalary()
Employee Number	computeTax()
Pan Number	
Salary	
Income Tax	

(b) Class Bank

Characteristics	Methods
Bank Name	openAccount()
Bank Address	depositMoney()
IFSC Code	
MICR Code	
Accounts	

(c) Class School

Characteristics	Methods
School Name	admitStudent()
School Address	conductExams()
Classes	
Students	
Teachers	

(d) Class Book

Characteristics	Methods
Book Name	buyBook()
ISBN Number	readBook()
Price	
Author	
Publisher	

(e) Class Park

Characteristics	Methods
Park Name	openPark()
Park Address	closePark()
Ticket Price	
Opening Time	
Closing Time	

(f) Class Medicine

Characteristics	Methods
Name	buyMedicine()
Quantity	sellMedicine()
Manufacturing Date	
Expiry Date	
Price	

(g) Class Computer

Characteristics	Methods
Model	startComputer()
Serial Number	shutdownComputer()
Processor	
RAM	
Hard Disk	

2. Object:

It is a basic unit of Object-Oriented Programming and represents the real-life entities. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated. An object has an identity, state, and behaviour. Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

3. Encapsulation:

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as **data-hiding**.

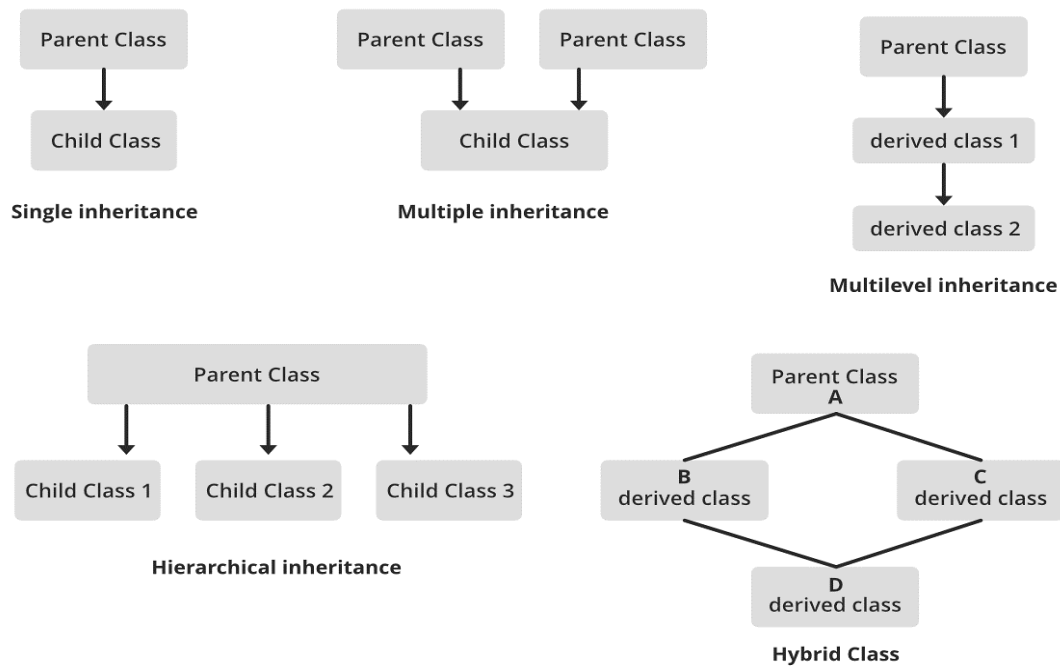
Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name "sales section".

4. Data Abstraction:

Data abstraction is one of the most essential and important features of object-oriented programming. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

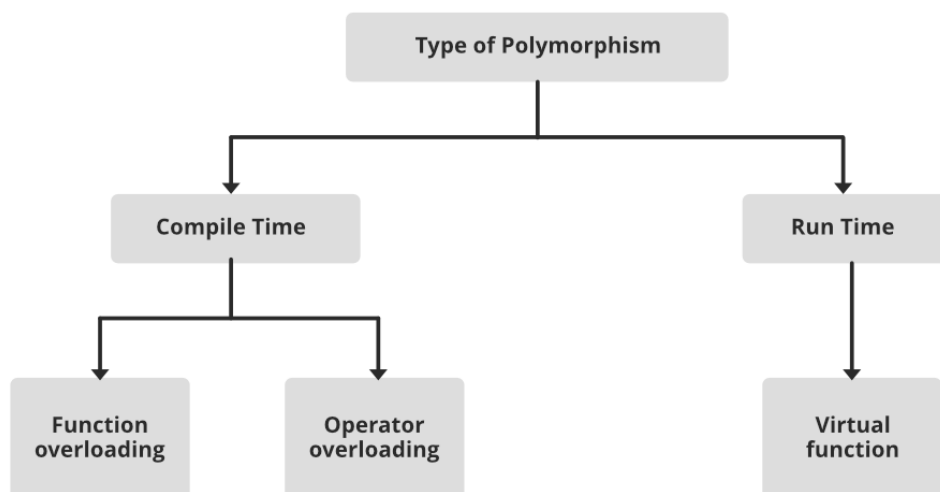
5. Inheritance:

Inheritance is an important pillar of OOP(Object-Oriented Programming). The capability of a class to derive properties and characteristics from another class is called Inheritance. When we write a class, we inherit properties from other classes. So, when we create a class, we do not need to write all the properties and functions again and again, as these can be inherited from another class that possesses it. Inheritance allows the user to reuse the code whenever possible and reduce its redundancy.



6. Polymorphism:

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. For example, A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So, the same person possesses different behaviour in different situations. This is called polymorphism.



7. Dynamic Binding:

In dynamic binding, the code to be executed in response to the function call is decided at runtime. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time. Dynamic Method Binding One of the main advantages of inheritance is that some derived class D has all the members of its base class B. Once D is not hiding any of the public members of B, then an object of D can represent B in any context where a B could be used. This feature is known as subtype polymorphism.

8. Message Passing:

It is a form of communication used in object-oriented programming as well as parallel programming. Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function, and the information to be sent.

Theoretical Questions and Answers on OOP Concepts

1. What is Object-Oriented Programming (OOP)?

Answer: Object-Oriented Programming is a programming paradigm based on the concept of "objects", which can contain data (attributes) and code (methods). OOP promotes better modularity, reusability, and scalability of code.

2. Explain the concept of Class and Object in OOP.

Answer:

- A **Class** is a blueprint or template for creating objects. It defines attributes and methods.
- An **Object** is an instance of a class. It has a state and behaviour defined by its class.

3. What is Encapsulation? Why is it important?

Answer: Encapsulation is the process of wrapping data and code into a single unit (class) and restricting access using access modifiers.

Importance:

- Enhances security
- Protects object integrity
- Allows controlled access to data

4. Define Inheritance. What are its types?

Answer: Inheritance allows one class (child/subclass) to acquire the properties and behaviour of another class (parent/superclass).

Types:

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance (not supported directly in Java)

5. What is Polymorphism? Explain its types.

Answer: Polymorphism means "many forms". It allows a single function or method to behave differently in different contexts.

Types:

- **Compile-time Polymorphism** (Method Overloading)
- **Run-time Polymorphism** (Method Overriding)

6. Explain Abstraction with an example.

Answer:

Abstraction is the process of hiding internal implementation details and showing only necessary features.

Example:

In a car, you use the steering wheel to turn but don't know the internal mechanical design.

7. What are the differences between Procedural Programming and Object-Oriented Programming?

Answer:

Procedural Programming	Object-Oriented Programming
Focuses on functions	Focuses on objects
Data is exposed to all	Data is hidden using encapsulation
No concept of inheritance	Supports inheritance
Less secure	More secure

8. What is the difference between abstraction and encapsulation?

Answer:

Feature	Abstraction	Encapsulation
Focus	Hiding complexity	Hiding data
Purpose	Show essential features	Protect internal data
Achieved by	Abstract classes, interfaces	Access modifiers, getters/setters

9. What is the role of access specifiers in OOP?

Answer: Access specifiers control the visibility of class members:

- **private** – accessible only within the class
- **protected** – accessible within the class and its subclasses
- **public** – accessible from anywhere
- **default (in Java)** – accessible within the package

10. Why is OOP important in software development?

Answer: OOP helps in:

- Organizing complex code
- Code reusability via inheritance
- Flexibility via polymorphism
- Security via encapsulation
- Scalability and maintainability

11. What is the "is-a" and "has-a" relationship in OOP?

Answer:

- **is-a:** Inheritance relationship (e.g., Dog is-a Animal)
- **has-a:** Composition relationship (e.g., Car has-a Engine)

12. What are real-life examples of OOP concepts?

Answer:

- **Encapsulation:** ATM machine (hides internal process)
- **Inheritance:** Animal → Dog, Cat
- **Polymorphism:** One interface (Remote), many devices (TV, AC)
- **Abstraction:** Using a phone without knowing internal circuits